

DOCUMENTAÇÃO DE PROJETO

AWS CLI

Instalação em Red Hat Linux, Configuração de Credenciais e Consulta Programática ao AWS IAM

Parâmetro	Detalhe
Ferramenta central	AWS CLI v2
Plataforma	Amazon EC2 — Red Hat Enterprise Linux
Escopo	Instalação manual, autenticação por chaves de acesso e consulta ao IAM via terminal
Duração aproximada	45 minutos

1. Problema / Contexto

Gerenciar recursos na AWS exclusivamente pelo Console de Gerenciamento é viável para operações pontuais, mas inviável para automação, scripts de infraestrutura e integração com pipelines de CI/CD. Qualquer tarefa que precise ser repetida, agendada ou executada de forma programática exige o uso da AWS CLI — a interface de linha de comando oficial que expõe os mesmos serviços do Console via terminal.

O ambiente deste projeto apresentou um desafio adicional: a instância EC2 utilizada rodava Red Hat Enterprise Linux, não Amazon Linux. Enquanto o Amazon Linux vem com a AWS CLI pré-instalada por padrão, distribuições como Red Hat, Ubuntu ou Debian exigem instalação manual. Saber realizar esse processo é fundamental para ambientes corporativos reais, onde a escolha do sistema operacional raramente é Amazon Linux.

Após a instalação e autenticação, o projeto incluiu um desafio prático: recuperar o documento JSON de uma política IAM usando apenas a linha de comando — sem acessar o Console Web. Esse cenário reproduz situações comuns em ambientes produtivos, onde o acesso ao Console pode ser restrito ou inexistente e toda a inspeção de configurações precisa ser feita via CLI.

2. Objetivo

Meu objetivo técnico foi estabelecer um ambiente de linha de comando completamente funcional em uma instância Red Hat Linux e utilizá-lo para interagir programaticamente com o AWS IAM. De forma específica:

- Instalar a AWS CLI v2 manualmente em uma instância EC2 com Red Hat Linux, seguindo o processo correto para distribuições sem a ferramenta pré-instalada.
- Configurar as credenciais de acesso via `aws configure`, compreendendo o papel de cada campo e o risco associado ao manuseio de chaves de acesso.

- Utilizar a CLI para consultar informações reais do AWS IAM, confirmando a autenticação e o funcionamento do ambiente.
- Resolver o desafio de recuperar o documento JSON de uma política IAM usando apenas comandos encadeados da CLI, sem acesso ao Console Web.

3. Solução

A solução foi executada em quatro etapas sequenciais: acesso à instância via SSH, instalação da AWS CLI, autenticação com credenciais IAM e uso da ferramenta para consultar e extrair dados do serviço IAM. A etapa de desafio — recuperar a política JSON via CLI — exigiu raciocínio sobre a estrutura dos dados retornados antes de formular o comando correto.

3.1 Acesso à Instância — SSH com Par de Chaves

O acesso à instância EC2 Red Hat foi feito via SSH utilizando um par de chaves no formato .pem. Em sistemas macOS e Linux, a permissão do arquivo de chave precisa ser restrita antes do uso — o SSH rejeita conexões com arquivos de chave com permissões abertas demais, por questão de segurança.

```
# Navegar ao diretório onde o arquivo de chave foi salvo
cd ~/Downloads

# Restringir as permissões do arquivo — obrigatório para SSH aceitar a chave
chmod 400 labsuser.pem

# Conectar à instância EC2 via SSH
ssh -i labsuser.pem ec2-user@<IP-DA-INSTANCIA>
```

Por que chmod 400 é obrigatório: O cliente SSH verifica as permissões do arquivo de chave privada antes de usá-la. Se o arquivo estiver acessível para outros usuários do sistema (permissões 644 ou 777, por exemplo), o SSH recusa a conexão com o erro 'Permissions are too open'. O chmod 400 restringe a leitura exclusivamente ao proprietário do arquivo.

3.2 Instalação da AWS CLI v2 em Red Hat Linux

Em distribuições que não incluem a AWS CLI por padrão, o processo de instalação segue três etapas: download do pacote oficial, extração e execução do instalador com privilégios de sistema. Cada etapa tem uma justificativa técnica específica.

```
# Etapa 1 — Baixar o instalador oficial diretamente do endpoint da AWS
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"

# Etapa 2 — Descompactar o pacote (-u evita confirmações para substituição de
arquivos)
unzip -u awscliv2.zip
```

```
# Etapa 3 – Executar o instalador com sudo (escrita em diretórios do sistema)
sudo ./aws/install

# Verificar a instalação confirmando a versão
aws --version

# Saída esperada
# aws-cli/2.x.x Python/3.x.x Linux/... botocore/...
```

Amazon Linux

AWS CLI pré-instalada. Nenhuma ação necessária — a ferramenta está disponível imediatamente após o login na instância.

Red Hat / Ubuntu / Debian

AWS CLI precisa ser instalada manualmente. O processo via curl + unzip + sudo install é o método oficial recomendado pela AWS para qualquer distribuição Linux.

3.3 Configuração das Credenciais — aws configure

Com a CLI instalada, o próximo passo foi autenticá-la em relação à conta AWS. O comando `aws configure` inicia um assistente interativo que armazena as credenciais e preferências de sessão em arquivos locais na instância (`~/.aws/credentials` e `~/.aws/config`).

Campo solicitado pelo aws configure	Valor configurado
AWS Access Key ID	[OCULTADO — dado sensível]
AWS Secret Access Key	[OCULTADO — dado sensível]
Default region name	us-west-2
Default output format	json

Chaves de acesso são credenciais de longa duração — manuseie com cuidado:

Diferentemente das credenciais temporárias geradas por Instance Profiles, chaves de acesso configuradas via `aws configure` ficam gravadas em `~/.aws/credentials` e não expiram automaticamente. Se a instância for comprometida ou o arquivo de credenciais vazar, o atacante herda acesso permanente à conta. Em produção, o padrão correto é usar Instance Profiles (sem chaves estáticas) ou credenciais temporárias via AWS STS.

3.4 Consulta ao IAM via CLI — Validação e Desafio

Com a autenticação configurada, validei o funcionamento da CLI listando os usuários IAM da conta. A resposta em JSON confirmou que a ferramenta estava autenticada e operacional:

```
# Listar todos os usuários IAM da conta – confirma autenticação funcional
aws iam list-users

# Retorno: objeto JSON com array de usuários, ARNs, datas de criação e IDs
```

O desafio proposto foi mais exigente: recuperar o documento JSON completo da política `lab_policy` usando apenas a CLI, sem acessar o Console Web. A solução exigiu dois comandos encadeados — o resultado do primeiro era entrada necessária para o segundo:

Passo	Comando	O que retorna
1	<code>aws iam list-policies --scope Local</code>	Lista de políticas do cliente com ARN e <code>DefaultVersionId</code> de cada uma
2	<code>aws iam get-policy-version --policy-arn <ARN> --version-id v1</code>	Documento JSON completo da política com todos os statements de permissão

```
# Passo 1 – Listar políticas gerenciadas pelo cliente para obter o ARN
# O filtro --scope Local exclui as milhares de políticas gerenciadas pela AWS
aws iam list-policies --scope Local

# Da saída, extraio: Arn e DefaultVersionId (ex.: v1)

# Passo 2 – Recuperar o JSON da política e salvar em arquivo local
aws iam get-policy-version \
  --policy-arn <ARN-DA-POLITICA> \
  --version-id v1 \
  > lab_policy.json

# Verificar o conteúdo extraído
cat lab_policy.json
```

Por que `--scope Local` é essencial no `list-policies`: Sem o filtro, o comando retorna centenas de políticas gerenciadas pela própria AWS (`AmazonS3ReadOnlyAccess`, `AmazonEC2FullAccess`, etc.), tornando a identificação da política correta extremamente difícil. O parâmetro `--scope Local` restringe o resultado apenas às políticas criadas pelo cliente na conta — geralmente poucas e com nomes específicos do ambiente.

4. Ferramentas

AWS CLI v2	AWS IAM	Amazon EC2
Red Hat Enterprise Linux	SSH / OpenSSH	curl
JSON (formato de saída)	Bash / Terminal	AWS STS (referência de boas práticas)

5. Resultado

A AWS CLI v2 foi instalada com sucesso em uma instância EC2 Red Hat Linux seguindo o processo manual correto. A autenticação via chaves de acesso foi configurada e validada — o comando `aws iam list-users` retornou a lista de usuários da conta em formato JSON, confirmando que a ferramenta estava autenticada e operacional.

O desafio de recuperar a política IAM foi concluído sem acesso ao Console Web. A execução dos dois comandos encadeados — `aws iam list-policies --scope Local` seguido de `aws iam get-policy-version` — resultou na criação do arquivo `lab_policy.json` com o documento completo da política, incluindo todos os statements de permissão em formato legível e exportável.

O ambiente resultante — uma instância Linux com CLI autenticada e funcional — reproduz a base de qualquer pipeline de automação de infraestrutura na AWS: a mesma configuração usada para scripts de provisionamento, automações de monitoramento e integrações com ferramentas como Ansible, Terraform e GitHub Actions.

6. Aprendizados

Este projeto consolidou três competências que se complementam: operar a linha de comando em um ambiente Linux não-padrão, autenticar programaticamente uma ferramenta em um serviço de nuvem e raciocinar sobre a estrutura dos dados retornados para formular consultas encadeadas. Cada uma dessas competências tem aplicação direta em ambientes produtivos.

6.1 Ambientes reais raramente são Amazon Linux — e isso muda o processo

O Amazon Linux foi construído para integrar nativamente com os serviços AWS — a CLI já vem instalada, o Instance Metadata Service está pré-configurado e várias otimizações de desempenho e segurança são específicas para essa distribuição. Mas em empresas que já possuem padronização em Red Hat, Ubuntu ou outra distribuição, o Amazon Linux não é uma opção.

Saber instalar a AWS CLI manualmente — e entender por que cada etapa do processo é necessária — é a diferença entre um profissional que só opera em ambientes pré-configurados e um que consegue configurar qualquer ambiente do zero. O processo `curl + unzip + sudo install` é o padrão oficial para qualquer distribuição Linux e deve ser parte do repertório de qualquer engenheiro que trabalhe com AWS.

6.2 Chaves de acesso estáticas são um risco que precisa ser gerenciado

O `aws configure` armazena as credenciais em texto simples no arquivo `~/.aws/credentials`. Essa conveniência tem um custo: as credenciais persistem indefinidamente na instância e não expiram automaticamente. Em laboratório controlado, isso é aceitável. Em produção, é um vetor de ataque.

O padrão seguro para instâncias EC2 é o Instance Profile — que associa um papel IAM diretamente à instância e fornece credenciais temporárias via metadados, sem nenhum arquivo de credenciais em disco. As credenciais são rotacionadas automaticamente a cada poucas horas pelo serviço de metadados. Nenhuma chave estática, nenhum arquivo para vaziar.

Hierarquia de preferência para credenciais na CLI: 1) Instance Profile (recomendado para EC2) → 2) Variáveis de ambiente `AWS_ACCESS_KEY_ID` e `AWS_SECRET_ACCESS_KEY` → 3) `aws configure` (arquivo `~/.aws/credentials`). Quanto mais alto na hierarquia, menor a superfície de exposição.

6.3 Consultas encadeadas: o padrão de trabalho real com CLI

O desafio de recuperar a política JSON ensinou algo que vai além do comando específico: o raciocínio de que muitas informações na AWS não estão disponíveis em um único endpoint. Para chegar ao conteúdo de uma política, é preciso primeiro descobrir o ARN e a versão — e para isso existe um comando separado. Essa estrutura de consultas encadeadas é o padrão de trabalho com a AWS CLI em praticamente qualquer serviço.

Scripts de automação profissionais fazem isso continuamente: buscam IDs dinamicamente com um comando e os passam como parâmetro para o próximo. Fixar ARNs, IDs de sub-rede ou IDs de AMI diretamente no script é um antipadrão que quebra quando o ambiente muda. O laboratório de Auto Scaling já havia demonstrado essa lógica na prática — este projeto reforçou o mesmo princípio com um serviço diferente.

6.4 O parâmetro `--scope Local` revela a importância de conhecer os filtros

Executar `aws iam list-policies` sem nenhum filtro retorna centenas de políticas gerenciadas pela AWS — tornando a identificação da política correta uma tarefa de agulha no palheiro. O parâmetro `--scope Local` não é um detalhe de sintaxe: é o filtro que torna o comando utilizável em um ambiente real.

Esse padrão se repete em toda a AWS CLI: comandos de listagem retornam conjuntos grandes de dados, e saber quais filtros e parâmetros aplicar é o que determina se o comando entrega o resultado esperado ou uma resposta inútil de centenas de linhas. Consultar a documentação com `aws iam list-policies help` antes de executar é o hábito que distingue quem usa a ferramenta com precisão de quem apenas tenta até funcionar.